One Proof to Rule Them All: Practical, Sublinear Verification for Actively Secure MPC on $\mathbb{Z}_{2^k}$ with Dishonest Majority and a Dealer

Andreas Brüggemann
brueggemann@encrypto.cs.tu-darmstadt.de
Technical University of Darmstadt
Darmstadt, Germany

Ariel Nof
ariel.nof@biu.ac.il
Bar-Ilan University
Ramat-Gan, Israel

Thomas Schneider
schneider@encrypto.cs.tu-darmstadt.de
Technical University of Darmstadt
Darmstadt, Germany

A ARTIFACT APPENDIX

A.1 Abstract

Our paper introduces a novel sublinear verification protocol for secure multiparty computation (MPC) in the n-party dishonest majority setting with a trusted dealer, working over a ring $\mathbb{Z}_{2^k}$. The artifact consists of the following MPC protocol implementations to securely evaluate arithmetic circuits: A) A passively secure MPC protocol with domain $\mathbb{Z}_{2^k}$ and function-dependent preprocessing, which acts as our baseline protocol. B) Our new actively secure MPC protocol, implementing and using our novel verification protocol on top of the baseline protocol to upgrade from passive to active security. C) Versions of both protocols that are generalized to also support computation of dot-products. Alongside the C++ implementations of the protocols, we provide a Docker container and scripts to benchmark the protocols for run time and communication on a single machine, locally emulating a network with up to 30 parties. We provide the artifact under the MIT license.[1]

A.2 Description & Requirements

Our MPC protocol implementations are located in the `src` directory. Centrally, `src/semi_evaluator.*` contains the passively secure baseline protocol, `src/zkfliop_evaluator.*` contains the actively secure protocol including the needed novel verification protocol, and the `_dotp` variants also support dot products.[2] The repository also contains benchmarking scripts (in the main directory and in `scripts`), our raw benchmark outputs used in the paper (`benchmark_results`, this also is the output target when re-running benchmarks), benchmark programs to construct benchmark circuits and run the protocols on, using synthetic input data as protocol performance is independent of input data (`benchmark`), tests (`test`), and material to compare to a selection of related works (`related_works`). Details can be found in `README.md`.

Optional Benchmarks for Related Works: In our paper, we also empirically compare to related works and their respective implementations. For that, we utilize these works' existing codebases with minor patches (located in `benchmark_results`) for a fair comparison. These codebases are *not* part of our artifact, and get cloned from their respective GitHub repositories, as they belong to different papers by different authors. We still make routines to benchmark and compare them available here, yielding *optional additional benchmarks* for related works. Alternatively, we also provide our measurements for them in `benchmark_results` as a fallback, noting that comparing these measurements to newly done benchmarks for our code will skew resulting comparisons due to potentially different system performances.

A.2.1 Security, privacy, and ethical concerns. As for our paper, we do not find any ethical concerns with our artifact. The provided docker container needs to be in privileged mode in order to allow to locally emulate a network for benchmarking, so it gains root access to the host system. Our Docker container and benchmark scripts for related works rely on third-party dependencies, theoretically opening the potential of supply chain attacks. We minimize risks by using dependencies either from standard Ubuntu repositories, or from GitHub in which case we pin a specific commit.

A.2.2 How to access. The artifact can be accessed on Zenodo: https://doi.org/10.5281/zenodo.22011889. Alternatively, the latest version is available on GitHub: https://encrypto.de/code/TheOneProof.

A.2.3 Hardware dependencies. For full support of all the artifact's function, an x86_64 architecture is required. For reproducing the results, at least 20 GB of RAM should be available on top of what is needed to run the host OS and other potentially running programs. Furthermore, we recommend to use a machine with at least 32 CPU hardware threads (usually 16 CPU cores), as the benchmarks run multiple parties on a single machine, each potentially using multiple threads, utilizing up to 32 threads. This is not a hard requirement, but less threads/cores will cause higher run times due to resource sharing. We require 4 GB of free disk storage.

A.2.4 Software dependencies. `docker` and `docker-buildx` are required to build and run the Docker container which takes care of all other dependencies.[3] Tested on Linux.

A.2.5 Benchmarks. No data requirements, all benchmarks run on synthetic data which is generated ad hoc.

A.3 Set Up

A.3.1 Installation. Download the artifact. Build and run the Docker container as follows (commands here for Linux, replace by the appropriate Docker commands if on other platform):

```
sudo docker buildx build --network=host -t zkfliop .
sudo docker run -it -v $(pwd):$(pwd) -w $(pwd) \
        --privileged zkfliop
```

For all the following steps, including compiling, tests, and evaluation, we require that they are executed from within the Docker container. Compile our code as follows (takes ca. 2 minutes):

[1] A single network emulation script, used for running benchmarks, is under GPL v3.
[2] The variant for dot products less optimizable, which is why we keep it separate.
[3] The dependencies automatically installed by the container are `Boost`, `clang`, `CMake`, `EMP Tool`, `git`, `GMP`, `g++`, `iproute2`, `libsodium`, `Matplotlib`, `Nlohmann JSON`, `NTL`, `OpenMP`, `OpenSSL`, `ping`, `Python3`.

`./compile.sh`

Optional step for comparing related works (takes ca. 10 minutes):

`./related_works_setup.sh`

A.3.2 Basic test. To test if the installation works as intended, run:

`./quicktest.sh`

This runs the actively secure protocol for 3 + 1 parties and prints the outputs of one of the parties. The expected behavior is that the output ends with "`ALL TESTS SUCCEEDED!`". Optionally, `./test.sh` can be used to run more extensive tests.

A.4 Evaluation Workflow

A.4.1 Major claims.

(C1): Our actively secure protocol, by using our sublinear verification protocol, has a communication overhead over the passively secure baseline that amortizes well over many multiplications, reaching only 0.7% overhead for $m = 10^6$ multiplications, at a round overhead of 19–28 rounds. This is proven by experiment (E1), with results reported in Tab. 3 and Tab. 4 of our paper.

(C2): Communication and computation of our verification, for $m = 10^6$ multiplications, lead to run time overheads (for active security) of $\approx$ 120%–270% in a LAN and $\approx$ 13%–75% in a WAN. This is proven by experiment (E1), with results reported in Tab. 3 of our paper.

(C3): Our actively secure protocol scales well with the number of parties n, yielding almost constant communication (except for very low n) and only a moderate increase or run time for larger n. This is proven by experiment (E1), with results reported in Fig. 10 of our paper.

(C4): Our actively secure protocol significantly improves over Asterisk, having $\approx 3.8\times$ better offline and $1.2\times$ better online communication, while especially improving offline run time by $\approx 30\times$ in a LAN and $\approx 9\times$ in a WAN. This is proven by experiment (E2) in conjunction with experiment (E1), with results reported in Tab. 5 of our paper.

A.4.2 Experiments.

(E1): **Benchmarking our protocols** [10 person-minutes + 2 compute-hours]: Automatically runs all benchmarks of our protocols (passive and active security), executing 240 individual sub-benchmarks in an emulated LAN and then repeating all 240 sub-benchmarks in an emulated WAN.
Preparation: None.
Execution: Run: `./reproduce.sh`
This will continuously update on the progress and will print "`All benchmarks done at [...]`" after finishing. Individual sub-benchmarks each should take few minutes at most.
Results: Run: `./parse_reproduced.sh E1`
This will print Tab. 3 and Tab. 4 of our paper which can be referenced to confirm claims (C1) and (C2). Furthermore, it writes what corresponds to Fig. 10 of our paper to `benchmark_results/plots/plot_parties_square.pdf`, confirming claim (C3). *Communication and rounds are expected to exactly match the values reported in the paper. Run times are also subject to compute performance which heavily depends on the used machine, which can create deviations from the values in the paper. These deviations naturally are expected to be less pronounced in a WAN, where communication becomes more critical than compute.*

(E2): **Optional Benchmarking of related works** [5 person-minutes + 30 compute-minutes]: This step automatically runs all benchmarks of related works, executing 15 individual sub-benchmarks for Asterisk each in LAN and WAN, and 20 individual sub-benchmarks in MP-SPDZ, each in LAN and WAN. This is an *optional step for comparing related works*: instead, it is also possible to skip the execution and follow the instructions for the results that use fallback data. The results require to run experiment (E1) in advance to generate our benchmarks to compare to.
Preparation: None.
Execution (optional, requires set up for related works from §A.3): Run: `./related_works_reproduce.sh`
This will continuously update on the progress and will print "`All related works benchmarks done at [...]`" after finishing. Individual sub-benchmarks each should take few minutes at most.
Results: If the optional related work benchmarks were executed, run: `./parse_reproduced.sh E2 relatedworks`
Else, for using fallback data: `./parse_reproduced.sh E2`
This will print Tab. 5 of our paper which can be referenced to confirm claim (C4). *Communication and rounds again should perfectly match the paper, whereas run times depend on available compute performance. Note that when using fallback data for Asterisk, run time data is compared between benchmarks on different machines, skewing the results.*

Note that running above experiments already generates all data displayed in the paper, also supporting many minor claims. This can be fully shown by running `./parse_reproduced.sh all`
or, if the optional related work benchmarks were executed,
`./parse_reproduced.sh all relatedworks`
The above instructions only print the data directly relevant to the major claims.

A.5 Notes on Reusability

Our artifact can be reused by using our protocol to securely evaluate other custom circuits. The repository contains a template in `test/template.cpp` for writing a custom wrapper to use our protocol. The template guides through defining a custom circuit and custom inputs to then run our MPC protocol on. This is enabled as the definition of the circuit to evaluate is decoupled from our MPC protocol evaluating this circuit.

Furthermore, the implementation of our verification protocol is thoroughly documented and may be used as a basis to implement similar verifications in other settings.

A.6 Version

Based on the LaTeX template for Artifact Evaluation V20220926.